Django Forms — Complete Notes

Level: Beginner to Advanced

Language: Bangla + English

Framework: Django

Topics: Forms, ModelForms, Validation, CRUD, Security, File Uploads, Testing, Best Practices

---

Table of Contents

1. Introduction to Django Forms
2. How Django Forms Work
3. Types of Django Forms
4. Creating a Basic Form
5. Form Fields
6. Widgets and HTML Attributes
7. Rendering Forms in Templates
8. GET and POST Requests
9. Form Validation
10. Custom Validation
11. Error Handling
12. ModelForm
13. Saving Data to Database
14. CRUD Operations with ModelForm
15. Updating Existing Objects
16. Initial Values and Disabled Fields
17. Dynamic Choices
18. File Uploads
19. Formsets
20. Authentication Forms
21. CSRF Protection
22. Form Security Best Practices
23. Class-Based Views
24. AJAX and JSON Responses
25. Testing Django Forms

26. Common Mistakes
27. Best Practices
28. Interview Questions
29. Practice Projects
30. Learning Roadmap

---

1. Introduction to Django Forms

What Is a Django Form?

Django Form is a built-in feature for collecting, validating, processing, and displaying user input.

Examples:

- Registration form
- Login form
- Contact form
- Search form
- Profile update form
- Product creation form
- File upload form

Why Do We Need Django Forms?

Without Django Forms, we need to handle HTML inputs, validation, errors, and submitted data manually.

Django Forms provide:

- Automatic HTML form rendering
- Input validation
- Error messages
- Data cleaning
- CSRF protection when used with Django's standard protections
- Integration with Django models
- Reusable form classes

Important: CSRF protection still requires correct template and view configuration. Django Forms alone do not automatically secure every request.

---

2. How Django Forms Work

The typical form-processing workflow is:

1. User opens the page.
2. Django displays an empty form.
3. User submits the form.
4. Django receives the submitted data.
5. Django validates the data.
6. Valid data becomes available through `cleaned_data`.
7. The application processes or saves the data.
8. Django redirects or displays errors.

Example:

```
if request.method == "POST":
    form = ContactForm(request.POST)

    if form.is_valid():
        name = form.cleaned_data["name"]
else:
    form = ContactForm()
```

Important Methods and Attributes

Method / Attribute	Purpose
`is_valid()`	Validates submitted data
`cleaned_data`	Contains validated field values
`errors`	Contains validation errors

`add_error()`	Adds a field-specific or non-field error
`clean()`	Validates multiple fields together
`clean_fieldname()`	Custom validation for one field
`as_p()`	Renders fields inside paragraphs
`as_div()`	Renders fields inside div elements
`save()`	Saves a ModelForm instance
`has_changed()`	Checks whether submitted

	values changed

`cleaned_data` should be accessed after successful validation.

---

3. Types of Django Forms

3.1 forms.Form

Use `forms.Form` when you need a form that is not directly tied to a database model.

Examples:

- Contact form
- Search form
- Filtering form
- Calculator input form

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

3.2 forms.ModelForm

Use `forms.ModelForm` when the form represents a Django model.

Examples:

- Create a product
- Update a profile
- Add a blog post
- Edit an employee record

```
from django import forms
from .models import Contact


class ContactModelForm(forms.ModelForm):
    class Meta:
```

```
        model = Contact
        fields = ["name", "email", "message"]
```

Form vs ModelForm

Feature	Form	ModelForm
Direct model integration	No	Yes
Automatic model field mapping	No	Yes
Built-in model-aware validation	No	Yes
Save method for model instance	No	Yes
Custom non-model input	Excellent	Possible
Typical use	Search, contact	CRUD operations

---

4. Creating a Basic Django Form

Assume the Django app is named `contact`.

Step 1: Create forms.py

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField(
        max_length=100,
        label="Your Name",
        widget=forms.TextInput(
            attrs={
                "class": "form-control",
                "placeholder": "Enter your name",
            }
        ),
    )

    email = forms.EmailField(
        label="Email Address",
        widget=forms.EmailInput(
            attrs={
                "class": "form-control",
                "placeholder": "Enter your email",
            }
        ),
    )

    message = forms.CharField(
        label="Message",
        widget=forms.Textarea(
            attrs={
                "class": "form-control",
                "rows": 4,
            }
        ),
    )
```

Explanation

- `CharField`: Accepts text.
- `EmailField`: Validates email format.

- `max_length`: Limits the maximum number of characters.
- `label`: Controls the displayed field label.
- `widget`: Controls how a field is rendered as HTML.
- `attrs`: Adds HTML attributes.

Step 2: Create the View

In `views.py`:

```
from django.shortcuts import render
from .forms import ContactForm


def contact_view(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
            name = form.cleaned_data["name"]
            email = form.cleaned_data["email"]
            message = form.cleaned_data["message"]

            # Process the submitted data here.

            return render(
                request,
                "contact/success.html",
                {"name": name},
            )
    else:
        form = ContactForm()

    return render(
        request,
        "contact/contact.html",
        {"form": form},
    )
```

Step 3: Configure URLs

In `contact/urls.py`:

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path("", views.contact_view, name="contact"),
]
```

In the project-level `urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("contact/", include("contact.urls")),
]
```

Step 4: Create the Template

File: `templates/contact/contact.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Contact Form</title>
</head>
<body>
    <h1>Contact Us</h1>

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Send Message</button>
    </form>
</body>
</html>
```

Step 5: Create the Success Template

File: `templates/contact/success.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Success</title>
</head>
<body>
    <h1>Thank you, {{ name }}!</h1>
    <p>Your message was submitted successfully.</p>
</body>
</html>
```

Step 6: Run the Server

```
python manage.py runserver
```

Open:
`http://127.0.0.1:8000/contact/`

---

5. Django Form Fields

Django provides multiple field types for validating different kinds of data.

Field	Purpose
`CharField`	Text
`EmailField`	Email address
`IntegerField`	Integer
`FloatField`	Floating-point number
`DecimalField`	Decimal number
`BooleanField`	Boolean value

`NullBooleanField`	Legacy nullable Boolean field; generally avoid in new code
`DateField`	Date
`DateTimeField`	Date and time
`TimeField`	Time
`DurationField`	Duration
`ChoiceField`	One choice
`MultipleChoiceField`	Multiple choices
`TypedChoiceField`	Choice converted to a Python type
`SlugField`	Slug-like text
`URLField`	URL
`UUIDField`	UUID

`RegexField`	Regex-based validation
`FileField`	Uploaded file
`ImageField`	Uploaded image
`JSONField`	JSON input
`GenericIPAddressField`	IP address

Example: Multiple Field Types

```
from django import forms


class ProductSearchForm(forms.Form):
    keyword = forms.CharField(
        required=False,
        max_length=100,
    )

    min_price = forms.DecimalField(
        required=False,
        min_value=0,
        max_digits=10,
        decimal_places=2,
    )

    available_only = forms.BooleanField(
        required=False,
    )
```

---

6. Required, Optional, and Default Values

Required Fields

By default, most form fields require a value.

```
name = forms.CharField(required=True)
```

Optional Fields

```
phone = forms.CharField(required=False)
```

Initial Values

```
country = forms.CharField(
    initial="Bangladesh",
)
```

Min and Max Values

```
age = forms.IntegerField(
    min_value=18,
    max_value=100,
)
```

Decimal Validation

```
price = forms.DecimalField(
    min_value=0,
    max_digits=10,
    decimal_places=2,
)
```

The browser may perform some validation, but server-side Django validation remains essential.

---

7. Widgets and HTML Attributes

Widgets determine how form fields are rendered in HTML.

Common Widgets

Widget	HTML element

`TextInput`	Text input
`EmailInput`	Email input
`PasswordInput`	Password input
`NumberInput`	Number input
`Textarea`	Multiline text
`CheckboxInput`	Checkbox
`Select`	Dropdown
`RadioSelect`	Radio buttons
`DateInput`	Date input
`DateTimeInput`	Date/time input
`FileInput`	File picker
`HiddenInput`	Hidden input

Example

```
from django import forms


class RegistrationForm(forms.Form):
    username = forms.CharField(
```

```
        max_length=50,
        widget=forms.TextInput(
            attrs={
                "class": "form-control",
                "placeholder": "Username",
                "autocomplete": "username",
            }
        ),
    )

    password = forms.CharField(
        widget=forms.PasswordInput(
            attrs={
                "class": "form-control",
                "autocomplete": "new-password",
            }
        ),
    )

    bio = forms.CharField(
        required=False,
        widget=forms.Textarea(
            attrs={
                "rows": 5,
                "placeholder": "Tell us about yourself",
            }
        ),
    )
```

Security note: `PasswordInput` masks the input visually. It does not hash or encrypt the submitted password. Use Django's authentication system for password handling.

---

8. Rendering Forms in Templates

Render the Entire Form

```
{{ form }}
```

Render Fields as Paragraphs

```
{{ form.as_p }}
```

Render Fields as Divs

```
{{ form.as_div }}
```

Render as a List

```
{{ form.as_ul }}
```

`as_p()`, `as_ul()`, and `as_table()` are familiar rendering methods. `as_div()` is available in modern Django versions. Check your installed version if a rendering method is unavailable.

Render Each Field Manually

```
<form method="post">
    {% csrf_token %}

    <div>
        {{ form.name.label_tag }}
        {{ form.name }}
        {{ form.name.errors }}
    </div>

    <div>
        {{ form.email.label_tag }}
        {{ form.email }}
        {{ form.email.errors }}
    </div>

    <button type="submit">Submit</button>
</form>
```

Manual rendering gives more control over layout and CSS.

Display Non-Field Errors

```
{{ form.non_field_errors }}
```

Display All Errors

```
{{ form.errors }}
```

---

9. GET and POST Requests

GET Request

A GET request usually displays an empty form.

```
form = ContactForm()
```

POST Request

A POST request carries submitted form data.

```
form = ContactForm(request.POST)
```

Standard Pattern

```
def contact_view(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
            # Process validated data.
            pass
    else:
        form = ContactForm()

    return render(
        request,
        "contact/contact.html",
        {"form": form},
    )
```

Why Use POST?

POST is appropriate when submitting data that creates or changes server-side state.

GET is generally appropriate for searches and filters that should be represented in the URL.

Neither method should be treated as encryption. Use HTTPS to protect data in transit.

---

10. Form Validation

Validation checks whether submitted data satisfies the application's requirements.

Basic Validation

```
if form.is_valid():
    print(form.cleaned_data)
```

```
else:
    print(form.errors)
```

How Validation Works

Django performs field conversion and validation, field-specific cleaning, and form-wide cleaning.

A simplified sequence is:

1. Raw input is received.
2. Values are converted into Python objects.
3. Field validators run.
4. `clean_<fieldname>()` methods run.
5. The form's `clean()` method runs.
6. Errors are collected.

If a field fails its earlier validation, its custom field-cleaning method may not run.

Example

```
from django import forms


class AgeForm(forms.Form):
    age = forms.IntegerField(min_value=18)
```

A submitted age below 18 fails validation.

Built-In Validators

```
from django import forms
from django.core.validators import MinLengthValidator


class UsernameForm(forms.Form):
    username = forms.CharField(
        validators=[
            MinLengthValidator(3),
        ],
    )
```

Validators can be reused across fields and models.

---

11. Custom Field Validation

Use `clean_<fieldname>()` to validate one field.

```
from django import forms


class RegistrationForm(forms.Form):
    username = forms.CharField(max_length=50)

    def clean_username(self):
        username = self.cleaned_data["username"]

        if len(username) < 3:
            raise forms.ValidationError(
                "Username must contain at least 3 characters."
            )

        return username
```

Rules

- The method name must match the field name.
- Raise `ValidationError` when validation fails.
- Return the cleaned value when validation succeeds.
- Access the value from `self.cleaned_data`.

---

12. Cross-Field Validation

Use `clean()` when validation depends on multiple fields.

Example: password confirmation.

```
from django import forms


class PasswordForm(forms.Form):
    password = forms.CharField(
        widget=forms.PasswordInput(),
    )

    confirm_password = forms.CharField(
        widget=forms.PasswordInput(),
    )

    def clean(self):
```

```
        cleaned_data = super().clean()

        password = cleaned_data.get("password")
        confirm_password = cleaned_data.get(
            "confirm_password"
        )

        if (
            password
            and confirm_password
            and password != confirm_password
        ):
            self.add_error(
                "confirm_password",
                "Passwords do not match.",
            )

        return cleaned_data
```

`add_error()` attaches an error to the specified field. Passing `None` as the field name creates a non-field error.

Important: This example only validates matching passwords. It does not create a user or safely store passwords.

---

13. ModelForm

A `ModelForm` generates form fields based on a Django model.

Step 1: Create a Model

In `models.py`:

```
from django.db import models


class Contact(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    message = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

Step 2: Create the ModelForm

In `forms.py`:

```
from django import forms
from .models import Contact


class ContactModelForm(forms.ModelForm):
    class Meta:
        model = Contact
        fields = [
            "name",
            "email",
            "message",
        ]
```

`Meta` Options

Option	Purpose
`model`	Model used by the form
`fields`	Fields included in the form
`exclude`	Fields excluded from the form
`labels`	Custom labels
`help_texts`	Help text for fields

`widgets`	Custom widgets
`error_messages`	Custom error messages
`field_classes`	Custom form field classes
`localized_fields`	Fields using localized formatting

Prefer explicitly listing permitted fields with `fields`.

Example with Custom Widgets

```
class ContactModelForm(forms.ModelForm):
    class Meta:
        model = Contact
        fields = ["name", "email", "message"]

        widgets = {
            "name": forms.TextInput(
                attrs={"class": "form-control"}
            ),
            "email": forms.EmailInput(
                attrs={"class": "form-control"}
            ),
            "message": forms.Textarea(
                attrs={"rows": 5}
            ),
        }

        labels = {
            "name": "Full Name",
            "email": "Email Address",
```

```
        }

        help_texts = {
            "name": "Enter your full name.",
        }
```

---

14. Saving ModelForm Data

Save Immediately

```
if form.is_valid():
    contact = form.save()
```

Save Without Committing

```
if form.is_valid():
    contact = form.save(commit=False)
    contact.name = contact.name.strip()
    contact.save()
```

`commit=False` returns an unsaved model instance.

For models with many-to-many relationships, call `form.save_m2m()` after saving the instance.

```
if form.is_valid():
    obj = form.save(commit=False)
    obj.owner = request.user
    obj.save()
    form.save_m2m()
```

Use this pattern when the model contains many-to-many fields that must be saved after the instance has a database identity.

---

15. Complete CRUD with ModelForm

CRUD means:

- Create
- Read
- Update
- Delete

Models

Use the `Contact` model from Section 13.

Forms

```
from django import forms
from .models import Contact


class ContactForm(forms.ModelForm):
    class Meta:
        model = Contact
        fields = ["name", "email", "message"]
```

URLs

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.contact_list, name="contact-list"),
    path("create/", views.contact_create, name="contact-create"),
    path("<int:pk>/edit/", views.contact_update, name="contact-update"),
    path("<int:pk>/delete/", views.contact_delete, name="contact-delete"),
]
```

Create

```
from django.shortcuts import render, redirect
from .forms import ContactForm


def contact_create(request):
    if request.method == "POST":
        form = ContactForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("contact-list")
    else:
        form = ContactForm()

    return render(
        request,
        "contacts/contact_form.html",
```

```
        {"form": form},
    )
```

Read

```
from django.shortcuts import render
from .models import Contact


def contact_list(request):
    contacts = Contact.objects.all().order_by("-created_at")

    return render(
        request,
        "contacts/contact_list.html",
        {"contacts": contacts},
    )
```

Update

```
from django.shortcuts import get_object_or_404
from .models import Contact


def contact_update(request, pk):
    contact = get_object_or_404(Contact, pk=pk)

    if request.method == "POST":
        form = ContactForm(
            request.POST,
            instance=contact,
        )

        if form.is_valid():
            form.save()
            return redirect("contact-list")
    else:
        form = ContactForm(instance=contact)

    return render(
        request,
        "contacts/contact_form.html",
```

```
        {"form": form},
    )
```

The `instance` argument tells Django to update the existing object instead of creating a new one.

Delete

```
def contact_delete(request, pk):
    contact = get_object_or_404(Contact, pk=pk)

    if request.method == "POST":
        contact.delete()
        return redirect("contact-list")

    return render(
        request,
        "contacts/contact_confirm_delete.html",
        {"contact": contact},
    )
```

Never delete a database record merely because a user opened a GET URL.

Form Template

`contacts/contact_form.html`:

```
<h1>Contact Form</h1>

<form method="post">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Save</button>
</form>
```

List Template

`contacts/contact_list.html`:

```
<h1>Contacts</h1>

<a href="{% url 'contact-create' %}">
    Create Contact
</a>
```

```
<ul>
    {% for contact in contacts %}
        <li>
            {{ contact.name }} — {{ contact.email }}

            <a href="{% url 'contact-update' contact.pk %}">
                Edit
            </a>

            <a href="{% url 'contact-delete' contact.pk %}">
                Delete
            </a>
        </li>
    {% empty %}
        <li>No contacts found.</li>
    {% endfor %}
</ul>
```

Delete Confirmation Template

`contacts/contact_confirm_delete.html`:

```
<h1>Delete Contact</h1>

<p>
    Are you sure you want to delete
    {{ contact.name }}?
</p>

<form method="post">
    {% csrf_token %}
    <button type="submit">Confirm Delete</button>
</form>

<a href="{% url 'contact-list' %}">Cancel</a>
```

In a real application, ensure users can access only records they are authorized to manage.

---

16. Initial Values and Disabled Fields

Initial Values

```
form = ContactForm(
    initial={
        "name": "Faruk",
        "email": "faruk@example.com",
    }
)
```

Initial values are useful for default display and should not be treated as trusted submitted data.

Updating with Initial Model Data

Prefer:

```
form = ContactForm(instance=contact)
```

for `ModelForm` updates.

Disabled Fields

```
class ProfileForm(forms.Form):
    username = forms.CharField(
        disabled=True,
    )
```

Django's disabled form field uses its initial value rather than accepting a submitted replacement as the field's value. Still, authorization must be enforced on the server independently.

---

17. Dynamic Choices

Suppose a form needs to display categories from the database.

```
from django import forms
from .models import Category


class ProductForm(forms.Form):
    category = forms.ModelChoiceField(
        queryset=Category.objects.none(),
    )

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
```

```
        self.fields["category"].queryset = (
            Category.objects.filter(is_active=True)
        )
```

`ModelChoiceField` validates that the submitted value corresponds to an object in the configured queryset.

For multiple selections, use `ModelMultipleChoiceField`.

Always limit the queryset according to the current user's permissions where applicable.

---

18. File Uploads

Form

```
from django import forms


class UploadForm(forms.Form):
    document = forms.FileField()
```

Template

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Upload</button>
</form>
```

View

```
def upload_document(request):
    if request.method == "POST":
        form = UploadForm(
            request.POST,
            request.FILES,
        )

        if form.is_valid():
            uploaded_file = form.cleaned_data["document"]

            # Process or save the uploaded file.
    else:
        form = UploadForm()
```

```
    return render(
        request,
        "upload.html",
        {"form": form},
    )
```

Image Upload

```
class ImageUploadForm(forms.Form):
    image = forms.ImageField()
```

`ImageField` requires Pillow.

```
pip install Pillow
```

File Upload Security

- Set reasonable upload-size limits.
- Validate allowed extensions and content.
- Do not trust the browser-provided MIME type alone.
- Store uploads safely.
- Avoid serving untrusted files as executable content.
- Consider malware scanning for higher-risk applications.
- Use private storage for confidential documents.

A valid file extension does not guarantee that a file is safe.

---

19. Formsets

A formset manages multiple instances of the same form on one page.

Use cases:

- Multiple product items
- Bulk editing
- Multiple addresses
- Repeated invoice lines

Example

```
from django import forms
from django.forms import formset_factory
```

```
class ItemForm(forms.Form):
    name = forms.CharField(max_length=100)
    quantity = forms.IntegerField(min_value=1)


ItemFormSet = formset_factory(
    ItemForm,
    extra=3,
    max_num=10,
    validate_max=True,
)
```

View

```
def item_view(request):
    if request.method == "POST":
        formset = ItemFormSet(request.POST)

        if formset.is_valid():
            items = formset.cleaned_data
            # Process the validated list.
    else:
        formset = ItemFormSet()

    return render(
        request,
        "items.html",
        {"formset": formset},
    )
```

Template

```
<form method="post">
    {% csrf_token %}
    {{ formset.management_form }}

    {% for form in formset %}
        {{ form.as_p }}
    {% endfor %}

    <button type="submit">Submit</button>
</form>
```

The management form is required because Django uses its hidden fields to track formset counts.

For database-backed formsets, explore `modelformset_factory()` and `inlineformset_factory()`.

---

20. Authentication Forms

Django provides built-in forms for authentication-related workflows.

Examples include:

- `AuthenticationForm`
- `UserCreationForm`
- `PasswordChangeForm`
- `PasswordResetForm`
- `SetPasswordForm`

Exact import paths and behavior may depend on the Django version and whether a custom user model is used.

Example: Login Form

```
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth import login
from django.shortcuts import render, redirect


def login_view(request):
    if request.method == "POST":
        form = AuthenticationForm(
            request,
            data=request.POST,
        )

        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect("dashboard")
    else:
        form = AuthenticationForm(request)

    return render(
        request,
        "accounts/login.html",
```

```
        {"form": form},
    )
```

Login Template

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Login</button>
</form>
```

Use Django's authentication system rather than writing custom password-storage logic. For production applications, consider login rate limiting, secure session settings, HTTPS, and appropriate authentication policies.

---

21. CSRF Protection

CSRF stands for Cross-Site Request Forgery.

Django provides CSRF middleware and template tags to protect unsafe requests.

Template

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Submit</button>
</form>
```

Important Rules

- Keep Django's CSRF middleware enabled.
- Include `{% csrf_token %}` in internal POST forms.
- Do not disable CSRF protection as a quick fix for errors.
- For JavaScript requests, configure the CSRF token according to Django's documentation.
- Protect state-changing actions, including updates and deletions.

CSRF protection is different from authentication and authorization.

---

22. Form Security Best Practices

22.1 Validate on the Server

Browser validation can be bypassed.

Always validate submitted values on the server.

22.2 Use Explicit ModelForm Fields

```
class Meta:
    model = Contact
    fields = ["name", "email", "message"]
```

Avoid unintentionally exposing privileged model fields.

22.3 Never Trust Client Input

Users can modify requests manually.

Do not trust hidden fields, disabled HTML inputs, IDs, prices, roles, or ownership values supplied by the browser.

22.4 Enforce Object Permissions

```
contact = get_object_or_404(
    Contact,
    pk=pk,
    owner=request.user,
)
```

This example assumes the model has an `owner` field. It restricts the object lookup to the current user.

22.5 Use ORM Queries Safely

Use Django ORM filtering rather than constructing SQL with raw user input.

22.6 Protect Passwords

Use Django's password hashing and authentication facilities.

Never store plaintext passwords.

22.7 Handle Errors Carefully

Avoid exposing database details, secrets, or stack traces to ordinary users.

22.8 Configure HTTPS

Use HTTPS in production and configure secure cookies and appropriate proxy settings for your deployment.

---

23. Class-Based Views

Django provides generic class-based views that reduce repetitive CRUD code.

Common views:

- `FormView`
- `CreateView`
- `UpdateView`
- `DeleteView`

CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Contact
from .forms import ContactForm


class ContactCreateView(CreateView):
    model = Contact
    form_class = ContactForm
    template_name = "contacts/contact_form.html"
    success_url = reverse_lazy("contact-list")
```

UpdateView

```
from django.views.generic import UpdateView


class ContactUpdateView(UpdateView):
    model = Contact
    form_class = ContactForm
    template_name = "contacts/contact_form.html"
    success_url = reverse_lazy("contact-list")
```

DeleteView

```
from django.views.generic import DeleteView


class ContactDeleteView(DeleteView):
    model = Contact
    template_name = "contacts/contact_confirm_delete.html"
    success_url = reverse_lazy("contact-list")
```

Configure URL patterns to point to each class's `.as_view()` method.
For user-owned records, override `get_queryset()` to enforce object-level authorization. Generic views do not automatically know your application's ownership rules.

---

24. AJAX and JSON Responses

Django Forms can also validate data submitted asynchronously.
Example:

```
from django.http import JsonResponse
from django.views.decorators.http import require_POST
from .forms import ContactForm


@require_POST
def contact_api(request):
    form = ContactForm(request.POST)

    if form.is_valid():
        return JsonResponse(
            {
                "success": True,
                "message": "Form submitted successfully.",
            }
        )

    return JsonResponse(
        {
            "success": False,
            "errors": form.errors.get_json_data(),
        },
        status=400,
    )
```

For JSON request bodies, parse and validate JSON separately or build an appropriate API serializer. `request.POST` is not automatically populated from an arbitrary JSON body.
For larger REST APIs, Django REST Framework serializers are often a better fit.

---

25. Testing Django Forms

Django forms should be tested for valid input, invalid input, and important edge cases.

Example Test

```
from django.test import TestCase
from .forms import ContactForm


class ContactFormTests(TestCase):

    def test_valid_form(self):
        form = ContactForm(
            data={
                "name": "Faruk",
                "email": "faruk@example.com",
                "message": "Hello Django",
            }
        )

        self.assertTrue(form.is_valid())

    def test_invalid_email(self):
        form = ContactForm(
            data={
                "name": "Faruk",
                "email": "invalid-email",
                "message": "Hello",
            }
        )

        self.assertFalse(form.is_valid())
        self.assertIn("email", form.errors)

    def test_missing_required_name(self):
        form = ContactForm(
            data={
                "name": "",
                "email": "faruk@example.com",
                "message": "Hello",
            }
        )
```

```
        self.assertFalse(form.is_valid())
        self.assertIn("name", form.errors)
```

Run tests:

```
python manage.py test
```

What Should You Test?

- Valid submissions
- Missing required fields
- Invalid email addresses
- Minimum and maximum values
- Custom validation
- Cross-field validation
- ModelForm saving and updating
- File-upload validation
- Authentication behavior
- Authorization and ownership
- CSRF behavior where relevant

---

26. Common Mistakes

Mistake 1: Forgetting `is_valid()`

Incorrect:

```
name = form.cleaned_data["name"]
```

Correct:

```
if form.is_valid():
    name = form.cleaned_data["name"]
```

Mistake 2: Forgetting `request.FILES`

Incorrect:

```
form = UploadForm(request.POST)
```

Correct:

```
form = UploadForm(
    request.POST,
    request.FILES,
)
```

Mistake 3: Missing CSRF Token

Incorrect:

```
<form method="post">
    {{ form.as_p }}
</form>
```

Correct:

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
</form>
```

Mistake 4: Creating a New Object Instead of Updating

Incorrect for an update:

```
form = ContactForm(request.POST)
```

Correct:

```
form = ContactForm(
    request.POST,
    instance=contact,
)
```

Mistake 5: Using `save()` on a Plain Form

A regular `forms.Form` does not provide the model-saving behavior of `ModelForm`.

Process `cleaned_data` explicitly or use a `ModelForm` when appropriate.

Mistake 6: Exposing Every Model Field

Prefer an explicit allowlist:

```
fields = ["name", "email", "message"]
```

Mistake 7: Deleting Data on GET

Use a confirmation page and perform deletion only on an authorized POST request.

---

27. Best Practices

1. Separate forms, models, views, and templates.
2. Use `forms.Form` for non-model input.
3. Use `forms.ModelForm` for model-backed CRUD.
4. Validate all untrusted input on the server.
5. Use `clean_<fieldname>()` for field-specific rules.
6. Use `clean()` for cross-field validation.
7. Display validation errors clearly.
8. Use explicit `ModelForm.Meta.fields`.
9. Use `get_object_or_404()` with suitable authorization filters.
10. Include CSRF protection in internal POST forms.
11. Use Post/Redirect/Get after successful form submissions when appropriate.
12. Use Django authentication tools for passwords.
13. Validate and safely store uploaded files.
14. Write tests for valid and invalid submissions.
15. Keep production secrets and debug settings secure.
16. Follow the documentation for your installed Django version.

---

28. Django Form Interview Questions

Beginner

Q1. What is a Django Form?

A Django Form collects user input, validates it, and provides cleaned data for processing.

Q2. What is the difference between Form and ModelForm?

`Form` is not automatically tied to a model. `ModelForm` derives fields and model-aware validation from a Django model and supports saving a model instance.

Q3. What does `is_valid()` do?

It triggers form validation and returns `True` if no validation errors exist.

Q4. What is `cleaned_data`?

It is a dictionary containing successfully cleaned field values.

Q5. Why use `forms.py`?

It provides a conventional location for reusable form definitions.

Intermediate

Q6. What is the purpose of `clean_<fieldname>()`?

It implements custom validation for one field.

Q7. What is the purpose of `clean()`?

It implements validation involving multiple fields.

Q8. What is `commit=False`?

It creates an unsaved model instance from a valid ModelForm.

Q9. Why is `instance=object` used?

It binds the form to an existing model object so that saving updates that object.

Q10. Why is `request.FILES` needed?

It provides uploaded file data to a form.

Q11. What is a formset?

A formset manages multiple instances of a form in one submission.

Advanced

Q12. What is the difference between field validation and form validation?

Field validation checks individual values; form-wide validation checks relationships or rules involving multiple fields.

Q13. How can you restrict a ModelForm queryset?

Override form initialization and set a filtered queryset on the relevant model-choice field.

Q14. How do you prevent unauthorized object updates?

Filter the target object or queryset by the current user's permissions and validate authorization on the server.

Q15. Why should a ModelForm use explicit fields?

It prevents unintended model fields from becoming editable through the form.

Q16. How do you test a Django Form?

Instantiate the form with test data and check `is_valid()`, `errors`, `cleaned_data`, and relevant persistence behavior.

Q17. How does Django handle CSRF?

CSRF middleware and tokens help verify that unsafe requests originate from an appropriately trusted context. They do not replace authentication or authorization.

---

29. Practice Projects

Beginner Projects

Project 1: Contact Form

Features:

- Name
- Email
- Message
- Required-field validation
- Success page

Project 2: Search Form

Features:

- Keyword
- Category
- Optional filters
- Search results

Intermediate Projects

Project 3: Student Management System

Features:

- Student creation
- Student list
- Student update
- Student deletion
- Field validation

Project 4: Blog Management

Features:

- Create posts
- Edit posts
- Category selection
- Draft and published status
- Author ownership

Project 5: Product Management

Features:

- Product name

- Price
- Description
- Category
- Image upload
- Validation and authorization

Advanced Projects

Project 6: User Profile System

Features:

- Profile editing
- Authentication
- Image upload
- Ownership checks
- Secure password handling

Project 7: Multi-Item Invoice Form

Features:

- Formsets
- Product choices
- Quantity validation
- Calculated totals
- Server-side price verification

Project 8: REST API Validation

Features:

- Django REST Framework serializers
- Request validation
- JSON error responses
- Authentication
- Automated tests

---

30. Learning Roadmap

Follow this order for a strong understanding of Django Forms.

Phase 1: Fundamentals

- Understand HTML forms.

- Learn GET and POST.
- Create `forms.py`.
- Learn basic fields.
- Render forms in templates.

Phase 2: Validation

- Use `is_valid()`.
- Understand `cleaned_data`.
- Display field errors.
- Write custom validators.
- Implement `clean_<fieldname>()`.
- Implement cross-field `clean()`.

Phase 3: Database Integration

- Understand Django models.
- Create ModelForms.
- Learn `save()`.
- Learn `commit=False`.
- Update objects using `instance=`.
- Build CRUD applications.

Phase 4: Advanced Forms

- Dynamic model-choice fields
- Formsets and model formsets
- File uploads
- Authentication forms
- Class-based views
- Asynchronous form submission

Phase 5: Production Quality

- CSRF protection
- Object-level authorization
- Secure file handling
- Error handling
- Automated tests
- Deployment configuration

---

31. Quick Revision Cheat Sheet

Basic Form

```
class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
```

Validate Form

```
if form.is_valid():
    data = form.cleaned_data
```

Custom Field Validation

```
def clean_name(self):
    name = self.cleaned_data["name"]
    return name.strip()
```

ModelForm

```
class ContactForm(forms.ModelForm):
    class Meta:
        model = Contact
        fields = ["name", "email"]
```

Save

```
if form.is_valid():
    form.save()
```

Update

```
form = ContactForm(
    request.POST,
    instance=contact,
)
```

File Upload

```
form = UploadForm(
    request.POST,
    request.FILES,
)
```

Template

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Submit</button>
</form>
```

Run Tests

```
python manage.py test
```

Conclusion

Django Forms are an essential part of building secure and maintainable Django applications.

The most important concepts to master are:

- `forms.Form`
- `forms.ModelForm`
- `is_valid()`
- `cleaned_data`
- `clean_<fieldname>()`
- `clean()`
- `save()` and `commit=False`
- `instance=`
- CSRF protection
- File validation
- CRUD operations
- Automated testing

Recommended next step: Build a complete Student Management System using Django ModelForm, including Create, Read, Update, Delete, custom validation, authentication, and tests.

For version-specific behavior, consult the official Django documentation: https://docs.djangoproject.com/en/stable/topics/forms/